[2026 NS] Taint Analysis

`taint_analysis.py` Homework: Dynamic Taint Analysis with Triton

Overview

Dynamic taint analysis tracks how untrusted data (e.g., user input, network packets) flows through a program at runtime. By marking external data as "tainted" and watching how it propagates through memory and registers, analysts can identify vulnerabilities like **buffer overflows** and **command injection**.

In this assignment, you will use **Triton** — a dynamic binary analysis framework — to perform taint analysis on a compiled Linux binary.

What is Taint Analysis?

Taint analysis answers the question: **"Can attacker-controlled data reach a sensitive operation?"**

There are three key concepts:

Concept	Meaning	In this assignment
Taint Source	Where untrusted data enters the program	`user_input` (simulates data from network/user)
Taint Propagation	How tainted data spreads through computations and copies	XOR transform, `strncpy` copy
Taint Sink	The sensitive location we want to check	`output_buf`

How the Script Works

The provided `taint_analysis.py` script uses Triton to **emulate** (simulate) the `vuln` binary instruction-by-instruction inside Python — it does NOT run the binary on the OS.

Since our script only loads the target binary (not libc), we must provide Python replacements ("hooks") for libc functions like `memset` and `strncpy`. Most hooks are already written for you.

Your job is to complete 3 TODOs related to the taint analysis itself:

Step	What happens	Your task
Load binary into Triton	Loads machine code into virtual memory	Already done
Hook libc functions	Replaces `memset`, `strncpy`, etc. with Python versions	**TODO 1**: implement `strncpy` with taint propagation
Mark taint source	Label `user_input` bytes as "tainted"	**TODO 2**
Emulate instructions	Triton executes each instruction and automatically tracks taint	Already done
Check taint sink	See if taint reached `output_buf`	**TODO 3**

Background: Function Calling Convention

On Linux 64-bit systems, function arguments are passed in registers:

Argument	Register
1st arg	`rdi`
2nd arg	`rsi`
3rd arg	`rdx`
Return value	`rax`

For example, when `strncpy(dest, src, n)` is called:

- `rdi` = dest, `rsi` = src, `rdx` = n

In Triton, you can read a register value with:

```
ctx.getConcreteRegisterValue(ctx.registers.rdi)
```

Environment Setup

> **Required OS: Linux (Ubuntu 22.04 or later)**
>
> This assignment requires a Linux x86-64 environment. If you are using macOS or Windows, please set up a Linux VM (e.g., VirtualBox, UTM, or WSL2).

```
# Install dependencies
sudo apt update && sudo apt install -y gcc python3 python3-pip

# Install Python packages
pip install triton-library lief

# Verify
python3 -c "from triton import *; print('Triton OK')"
python3 -c "import lief; print('LIEF OK')"
```

Compile the Target Binary

```
gcc -o vuln vuln.c -no-pie -fno-stack-protector -fcf-protection=none
```

Target Program: `vuln.c`

The following program simulates a simple data processing pipeline. `user_input` represents **untrusted external input** (e.g., data received from a network socket). In a real application this would come from `recv()` or `fgets()`; here we pre-initialize it with `"AAA..."` for simplicity.

Our goal: **check if attacker-controlled data (`user_input`) can reach `output_buf` through the program's data flow.**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Taint source: simulates received network/user data
char user_input[64] = "AAAAAAAAAAAAAAAA";  // 16 bytes of input data

char processed[64];  // intermediate buffer
char output_buf[64]; // taint sink

void process_data() {
  int i;
  for (i = 0; i < 16; i++) {
    processed[i] = user_input[i] ^ 0x42;  // XOR transform
  }
}

void copy_to_output() { strncpy(output_buf, processed, 16); }

int main() {
  memset(processed, 0, 64);  // clear intermediate buffer
  memset(output_buf, 0, 64); // clear output buffer
  process_data();
```

```
  copy_to_output();
  printf("Output: %s\n", output_buf);
  return 0;
}

```

Data flow: `user_input` → XOR transform → `processed` → `strncpy` → `output_buf`

Your Task

Complete the provided `taint_analysis.py` script by filling in the **3 TODO sections**.

TODO #	Location	What to implement
1	`hook_strncpy`	Copy bytes src→dest and **propagate taint** for each byte
2	`emulate` (taint source)	Mark each byte of `user_input` as tainted
3	`emulate` (taint sink)	Check each byte of `output_buf` for taint and print the result

Hints

- **TODO 1**: Look at `hook_memset` (already implemented) for a reference on how to read function arguments and access memory. The key difference: `strncpy` copies data from `src` to `dest`, so you must also **copy the taint status** of each byte.
- **TODO 2**: Use `ctx.taintMemory(addr)` in a loop.
- **TODO 3**: Use `ctx.isMemoryTainted(addr)` in a loop, print whether each byte is `[TAINTED]` or `[CLEAN]`.

Expected Output

When your script works correctly, you should see output similar to this:

```
==================================================
  TAINT ANALYSIS WITH TRITON
==================================================
  Source: user_input @ 0x404060
  Sink:   output_buf @ 0x404100
  Track length: 16 bytes
==================================================
```

```

  [TAINTED] byte 0 at 0x404100
  [TAINTED] byte 1 at 0x404101
  [TAINTED] byte 2 at 0x404102
  ...
  [TAINTED] byte 15 at 0x40410f

Result: 16 / 16 bytes tainted at sink

Done.

```

Key check: All 16 bytes of `output_buf` should be marked as `[TAINTED]`.

> **Note**: The memory addresses (e.g., `0x404060`, `0x404100`) may differ slightly depending on your GCC version. This is normal — the script resolves addresses automatically. Focus on whether all 16 bytes show `[TAINTED]`.

Submission

Submit **ONE zip file** named `<StudentID>_taint_hw.zip` containing:

File	Description
`taint_analysis.py`	`The modified Python script containing your implemented code for TODO 1, 2, and 3`
`output.txt`	The full terminal output when running `python3 taint_analysis.py` (you can generate this via `python3 taint_analysis.py > output.txt`)
`report.pdf`	Report answering the 4 questions below

Report Questions (60 pts)

Answer the following in your report:

1. **(15 pts)** Explain what happens at each stage of the taint propagation path: `user_input` → `process_data()` → `processed` → `strncpy` → `output_buf`. Why does the XOR

operation (`processed[i] = user_input[i] ^ 0x42`) propagate taint? Why does XOR with a constant NOT remove taint?

2. **(15 pts)** In your `hook_strncpy` implementation, you manually propagated taint byte-by-byte. **Why is this necessary?** What would happen if you did NOT implement taint propagation in the hook? (Hint: Triton only tracks taint through instructions it emulates, not through hooked functions.)
3. **(15 pts)** Give a **real-world security vulnerability scenario** (e.g., SQL injection, XSS, or buffer overflow) and explain how taint analysis could help detect it. Describe what would be the taint source, taint sink, and the expected propagation path.
4. **(15 pts)** Discuss at least **two limitations** of dynamic taint analysis in practical applications. What complementary techniques or tools could be used to address these limitations?

Grading Summary

Component	Points
TODO 1–3 correctly implemented (script runs and produces correct output)	40
Report Question 1	15
Report Question 2	15
Report Question 3	15
Report Question 4	15
Total	**100**

For questions, please contact TA 蕭旭傑 via E3 platform, or email <shjay629.cs13@nycu.edu.tw>

taint_analysis.py
01 May 2026, 01:05 AM

vuln.c
01 May 2026, 01:05 AM

References

1. Triton: Triton: A dynamic binary analysis library
2. Triton Python API — TritonContext: libTriton: TritonContext
3. Triton GitHub & Examples: Triton/src/examples/python at master · JonathanSalwan/Triton
4. Forward Tainting Example: Triton/src/examples/python/forward_tainting.py at master · JonathanSalwan/Triton
5. LIEF: LIEF

利用 Triton: A dynamic binary analysis library 來追蹤程式內1資訊流

Taint Analysis | r809's Notes